FEAT-ZD-48600 · DE-04 Identify the sources/pipelines to be changed (Ingestion) ■ · Components, layers, sources or pipelines to change

Data Engineering agent · item task.components · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.components.md · approved by Data engineer sim.data_engineer

task.components: FEAT-ZD-48600

Sources and pipelines to create or update in the **ingestion layer** for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-04, Identify the sources/pipelines to be changed (Ingestion), attempt 1. There is no gate: automated checks, the independent judge and sampled human evaluation review this output.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-04 Identify the sources/pipelines to be changed (Ingestion), attempt 1	record:brief
Inputs	feat.tasks (b0fd4d2), feat.design (722a98f), org.data_model (a6b3e2a), task.repo_access (c7b4be8)	record:brief
Background read	journey:task.secrets_scope (DE-02), journey:task.db_changes (DE-03)	journey:task.secrets_scope; journey:task.db_changes
Code read	ap_pipeline source and tests, as given in the step's tool view	tool:repo
Repository / branch	ap_pipeline, branch `feature/story` at `3fcbe1b`, write access confirmed	task.repo_access (P-5, P-6)
Data Engineering tasks in scope	TSK-01 (STORY-01, optional `paid_date`), TSK-02 (STORY-02, `delays_by_vendor`)	feat.tasks (section 1)
Result	**No ingestion source or ingestion pipeline needs to be created or changed.** The existing CSV ingestion already carries the new optional `paid_date` column. Every change for this feature sits in the transformation layer (`transform.py`) and in one new derived-data module (`delays.py`)	tool:repo; feat.design (HC-3, B1.1, B1.3); feat.tasks (TSK-01, TSK-02)

1. What exists today (read from the repository)

Element	Layer	What it does today	Source
Vendor invoice feed	Source	CSV text with a header row. The tests' inline feed uses today's (v2) header `invoice_id,vendor,due_date,amount,paid`	tool:repo (`ingest.py`, `tests/test_pipeline.py`)
`ingest.parse_feed(csv_text)`	Ingestion	`[dict(r) for r in csv.DictReader(io.StringIO(csv_text))]`: one `dict` of strings per row, keyed by the header text	tool:repo (`ingest.py`)

Element	Layer	What it does today	Source
`transform.to_model(rows)`	Transformation	Maps rows to the Invoice model (`id`, `vendor`, `due_date`, `amount`, `paid`); a row with a missing or invalid required field is rejected with the exception type name as `reason`	tool:repo (`transform.py`)
`dq.quality_report(records, rejected)`	Data quality	`rows`, `loaded`, `rejected`, and the sorted set of `reject_reasons`	tool:repo (`dq.py`)
`pipeline.run(csv_text)`	Pipeline (orchestration)	`to_model(parse_feed(csv_text))`, then returns `(records, quality_report(...))`	tool:repo (`pipeline.py`)
`schedule.json`	Schedule	`daily`, `02:00`, `Asia/Kolkata`	tool:repo (`schedule.json`)
Package version	Metadata	`0.4.0`	tool:repo (`__init__.py`); feat.tasks (section 2)

Databricks, workspaces and databases are not used by ap_pipeline (task.repo_access P-1; journey:task.db_changes section 4).

2. Ingestion-layer change list (the outcome of this step)

#	Source / pipeline	Action	Lines of code to change	Why this is the minimum	Source
IN-1	Vendor invoice feed: v2 → v3 (adds the optional column `paid_date`, an ISO date or blank)	**Update the source contract only; no code.** Ingestion must keep accepting v2 (no `paid_date` column) and accept v3	None	`csv.DictReader` turns every header column into a key, so a v3 `paid_date` column arrives as a key with no code change. A v2 feed simply has no such key, and a short v3 row gives the value `None`. After TSK-01, `to_model` will read it with `r.get("paid_date")`, which covers both	org.data_model ("paid_date added in feed v3"); feat.design (B1.3 "Feed row", NF-4); feat.tasks (TSK-01 implementation note 1); tool:repo (`ingest.py`)
IN-2	`ingest.parse_feed`	**No change**	None	Already header-driven (IN-1); feat.tasks lists `ingest.py` under TSK-01 "Must not change"	feat.tasks (TSK-01); feat.design (HC-3, B1.1); tool:repo

#	Source / pipeline	Action	Lines of code to change	Why this is the minimum	Source
IN-3	`pipeline.run`	**No change**	None	Its signature and `(records, report)` return stay as they are. After TSK-01, records gain the `paid_date` key inside `to_model`, and the new reject reason `'paid_date'` reaches `reject_reasons` through `dq.quality_report` unchanged. `delays_by_vendor` is called by the caller (the end-to-end harness), not by `pipeline.run`	feat.design (HC-3, A4, B5); feat.tasks (TSK-01 "Must not change"); tool:repo (`pipeline.py`, `dq.py`)
IN-4	`dq.quality_report`	**No change**	None	It builds `reject_reasons` from each rejected row's `reason`, so `'paid_date'` appears without a code change	feat.design (HC-3); tool:repo (`dq.py`)
IN-5	`schedule.json`	**No change** (daily, 02:00, Asia/Kolkata)	None	On TSK-01 "Must not change"; the design keeps the feed cadence	feat.tasks (TSK-01); feat.design (B1.1, A4)
IN-6	New ingestion source or pipeline	**None created**	None	No new repository, service, runtime, library or storage (DD-1). Storage, database and schema migration are out of the design	feat.design (DD-1, A1); journey:task.db_changes (section 4)

Summary for the ingestion layer: no source or pipeline code to create or update. Supporting feed v3 needs no ingestion code. Its only ingestion-side effect is a change to the source contract (IN-1), which is already in the registered data model.

3. Components outside the ingestion layer (for the later Data Engineering steps)

The ingestion reading above shows where the feature's code lands. These items are listed so that the later Data Engineering steps (data model, transformation, coding) start from one list; they are taken from the task budgets, not decided here.

#	Component	Layer	Action	Task / design	Files in budget	Source
TC-1	`transform.to_model` + private `_parse_paid_date` + constant `PAID_DATE_REASON = "paid_date"`	Transformation	Update (additive). Keep the existing `try` block as it is; parse `paid_date` only for a row that passed it; reject a non-blank invalid value with reason `'paid_date'`; add `paid_date` (a `date` or `None`) to every record	TSK-01; HC-1; B1.2	`src/ap_pipeline/transform.py`; `src/ap_pipeline/__init__.py` (version step only); `tests/test_pipeline.py`	feat.tasks (TSK-01); feat.design (HC-1, B1.2, DD-6, DD-7)
TC-2	`delays.delays_by_vendor(records)`	Derived data (ap_pipeline)	Create (new module, pure, no I/O)	TSK-02; HC-2; B1.2	New `src/ap_pipeline/delays.py`; new `tests/test_delays.py`	feat.tasks (TSK-02); feat.design (HC-2, B1.2)

Rules carried for coding: keep the `to_model` signature and the `(records, rejected)` return shape; do not add `strip()` to `due_date` parsing; no `__all__` in `ap_pipeline/__init__.py` (import as `from ap_pipeline.delays import delays_by_vendor`); amounts stay `Decimal` (feat.tasks TSK-01, TSK-02; record:memory D-2026-044, L-011).

4. Secrets access in all environments

Environment	Secrets required for ingestion	Granted	Gap	Source
Local development and CI (`/usr/bin/python3 ci.py` on `feature/story`)	None	None	None	journey:task.secrets_scope (section 2); task.repo_access (P-7)
Integration (`dev`) and released (`main`)	None	None	None	journey:task.secrets_scope (section 2)

`pipeline.run` takes the feed as `csv_text`; the repository holds no feed connection, credential or workspace (tool:repo; task.repo_access P-1). Coding builds on synthetic v3 rows, so it needs no feed credential. If releasing feed v3 needs one, DE-02 runs again on that re-entry trigger (journey:task.secrets_scope SS-1; feat.tasks TSK-01 "Depends on").

5. Data model check

The registered model already reads: Invoice `id` (str), `vendor` (str), `due_date` (date), `amount` (Decimal, 2 dp), `paid` (bool), `paid_date` (date or None), "paid_date added in feed v3" (org.data_model). This matches DC-1 in feat.design (A3, B1.3). The ingestion layer adds no further field, so this step finds no conflict for DE-05.

6. Dependencies and risks for the ingestion layer

ID	Item	Effect	Handling	Source
R-I1	DT-1 (TSK-01) needs feed v3 in production, while ZD-48470 brings feed v2 (PQ-6)	Gates release of TSK-01, not coding	Ingestion is header-driven, so v2 keeps loading and v3 needs no ingestion change; tests use synthetic v3 rows	feat.design (Part C "External dependencies"); feat.tasks (TSK-01 "Depends on")

ID	Item	Effect	Handling	Source
R-I2	`parse_feed` keys rows by the exact header text, and TSK-01 reads `r.get("paid_date")`	Agent judgement from these two facts: a v3 header that spells the column differently would give every record `paid_date = None` with no reject	Open question CQ-I1	tool:repo (`ingest.py`); feat.tasks (TSK-01 implementation notes 1 and 2)
R-I3	No input names the production caller that supplies `csv_text` and calls the stages	Where the feed is fetched from is not known to this step	Carried as DQ-1; no change here	feat.design (A4, DQ-1); tool:repo (`pipeline.py`)

7. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
CQ-I1	Confirm that the feed v3 header names the new column exactly `paid_date` (lower case, no spaces). The inputs name the field but give no v3 header specification	Assume `paid_date` as in org.data_model and feat.design. Agent judgement: the TSK-01 tests that use a `paid_date` column should use the full v3 header	Data engineering lead / feed owner
CQ-I2	Carried DQ-1: which production component fetches the feed and calls `pipeline.run`?	Not needed for ingestion code; only the end-to-end harness wires the stages in this feature	Architect

8. Validation check

Check	Result	Source
Existing sources and pipelines read	Pass: `ingest.py`, `transform.py`, `dq.py`, `pipeline.py`, `schedule.json`, `__init__.py`, `tests/test_pipeline.py`	tool:repo
Ingestion sources/pipelines to create or update listed	Pass: none to create; source contract v2 → v3 with no code change (section 2)	feat.design (HC-3); tool:repo
Minimum lines of code	Pass: no ingestion line changed; feature code limited to the TSK-01 and TSK-02 budgets	feat.tasks (TSK-01, TSK-02)
Secrets in all environments	Pass: none required, none granted, no gap	journey:task.secrets_scope
Production or source-system data read	None; only inputs, the tool view and synthetic test data	record:brief
Conflict with an approved output	None	feat.design; feat.tasks; journey:task.secrets_scope; journey:task.db_changes
Blocking questions	None	—